

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 1
За седьмой семестр
По дисциплине «Интеллектуальный анализ данных»
Тема: «РСА»

Выполнил:
Студент 4 курса
Группы ИИ-26
РулькоМ.А.
Проверил:
Никитин С.К.

Цель работы:

- научиться применять метод PCA для осуществления визуализации данных

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент (двумя способами: 1. вручную через использование `numpy.linalg.eig` для вычисления собственных значений и собственных векторов и 2. с помощью `sklearn.decomposition.PCA` для непосредственного применения метода PCA – два независимых варианта решения);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки `matplotlib`, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Используя собственные значения, рассчитанные на этапе 1, вычислить потери, связанные с преобразованием по методу PCA. Сделать выводы;
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№ варианта	Выборка	Класс
1	seeds.zip	Последняя колонка

Ход работы

Код программы:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

class SeedPCAAalyzer:
    def __init__(self, data_path: str = "seeds_dataset.txt"):
        self.data_path = data_path
        self.column_names = [
            "area",
            "perimeter",
            "compactness",
            "kernel_length",
            "kernel_width",
            "asymmetry_coefficient",
            "kernel_groove_length",
            "class",
        ]
        self.colors = {1: "red", 2: "green", 3: "blue"}
        self.data, self.target = self._load_data()

    def _load_data(self):
        """Загрузка данных и заполнение пропусков средними значениями."""
```

```

dataset = pd.read_csv(
    self.data_path, sep=r"\s+", header=None, names=self.column_names
)
dataset.fillna(dataset.mean(numeric_only=True), inplace=True)

y = dataset["class"].to_numpy(dtype=int)
X = dataset.drop(columns=["class"]).to_numpy(dtype=float)
return X, y

def compute_custom_pca(self, n_components: int):
    """Ручная реализация PCA через ковариационную матрицу."""
    # Центрирование данных
    centered_X = self.data - np.mean(self.data, axis=0)

    # Вычисление ковариационной матрицы и собственных векторов
    cov_mat = np.cov(centered_X, rowvar=False)
    e_vals, e_vecs = np.linalg.eig(cov_mat)

    # Сортировка по убыванию собственных значений
    sorted_indices = np.argsort(e_vals)[::-1]
    e_vals_sorted = e_vals[sorted_indices].real
    e_vecs_sorted = e_vecs[:, sorted_indices].real

    # Проекция на компоненты
    W = e_vecs_sorted[:, :n_components]
    transformed = np.dot(centered_X, W)

    # Расчет потери информации в процентах
    loss_pct = 100.0 * (1.0 - np.sum(e_vals_sorted[:n_components]) / np.sum(e_vals_sorted))
    return transformed, loss_pct

def compute_sklearn_pca(self, n_components: int):
    """Реализация PCA с помощью библиотечного класса scikit-learn."""
    pca_model = PCA(n_components=n_components)
    transformed = pca_model.fit_transform(self.data)

    loss_pct = 100.0 * (1.0 - np.sum(pca_model.explained_variance_ratio_))
    return transformed, loss_pct

def _draw_scatter(self, ax, projected, title, is_3d=False):
    """Вспомогательный метод для отрисовки точек на графике."""
    for label in np.unique(self.target):
        indices = np.where(self.target == label)
        color = self.colors.get(label, "gray")

        if is_3d:
            ax.scatter(
                projected[indices, 0],
                projected[indices, 1],
                projected[indices, 2],
                c=color,
                label=f"Класс {label}",
                s=25,
            )
        else:
            ax.scatter(
                projected[indices, 0],

```

```

        projected[indices, 1],
        c=color,
        label=f"Класс {label}",
        s=25,
    )

```

```

ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
if is_3d:
    ax.set_zlabel("PC3")
ax.set_title(title)
ax.legend()

```

```

def visualize_all(self, custom_2d, sklearn_2d, custom_3d, sklearn_3d):
    """Отрисовка всех результатов на 2D и 3D графиках."""
    # 2D Сравнение
    fig_2d, axes_2d = plt.subplots(1, 2, figsize=(13, 5))
    self._draw_scatter(axes_2d[0], custom_2d, "2D PCA (Ручная реализация)")
    self._draw_scatter(axes_2d[1], sklearn_2d, "2D PCA (Scikit-Learn)")
    fig_2d.tight_layout()
    fig_2d.savefig("pca_2d.png", dpi=150)

    # 3D Сравнение
    fig_3d = plt.figure(figsize=(13, 5))
    ax_3d_custom = fig_3d.add_subplot(1, 2, 1, projection="3d")
    ax_3d_sklearn = fig_3d.add_subplot(1, 2, 2, projection="3d")

    self._draw_scatter(ax_3d_custom, custom_3d, "3D PCA (Ручная реализация)", is_3d=True)
    self._draw_scatter(ax_3d_sklearn, sklearn_3d, "3D PCA (Scikit-Learn)", is_3d=True)

    fig_3d.tight_layout()
    fig_3d.savefig("pca_3d.png", dpi=150)

    plt.show()

```

```

def main():
    analyzer = SeedPCAAnalyzer(data_path="seeds_dataset.txt")

    # Вычисления для ручного метода
    custom_2d, loss_custom_2d = analyzer.compute_custom_pca(n_components=2)
    custom_3d, loss_custom_3d = analyzer.compute_custom_pca(n_components=3)

    # Вычисления для метода sklearn
    sklearn_2d, loss_sklearn_2d = analyzer.compute_sklearn_pca(n_components=2)
    sklearn_3d, loss_sklearn_3d = analyzer.compute_sklearn_pca(n_components=3)

    # Вывод потерь информации
    print("--- Результаты потерь информации ---")
    print(f"Ручной подход (2D): {loss_custom_2d:.2f}%")
    print(f"Ручной подход (3D): {loss_custom_3d:.2f}%")
    print(f"Sklearn (2D): {loss_sklearn_2d:.2f}%")
    print(f"Sklearn (3D): {loss_sklearn_3d:.2f}%")

    # Визуализация
    analyzer.visualize_all(custom_2d, sklearn_2d, custom_3d, sklearn_3d)

```

```
if __name__ == "__main__":  
    main()
```

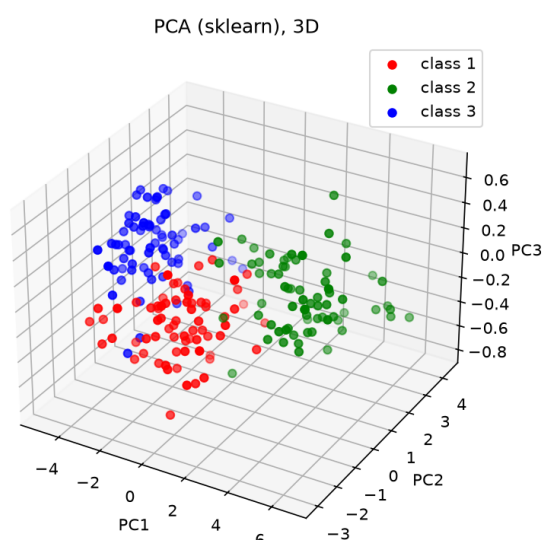
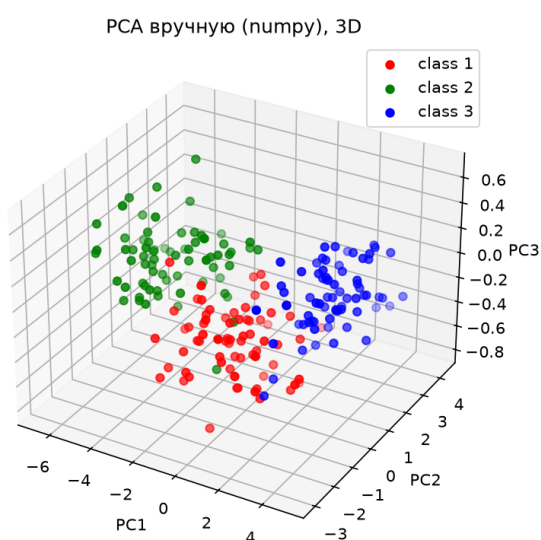
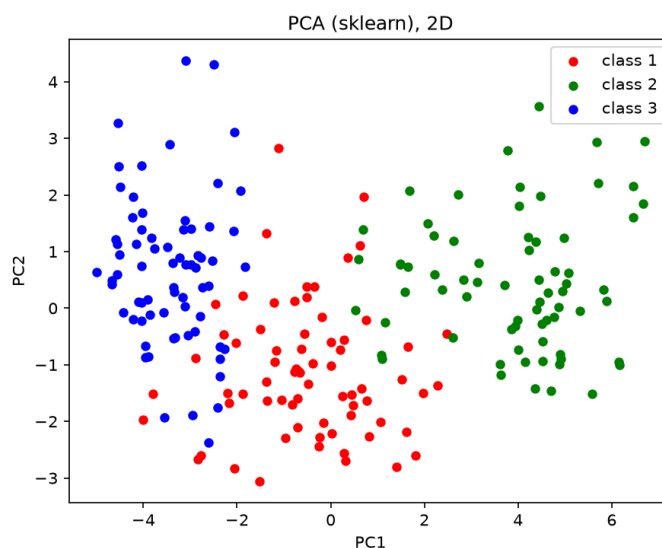
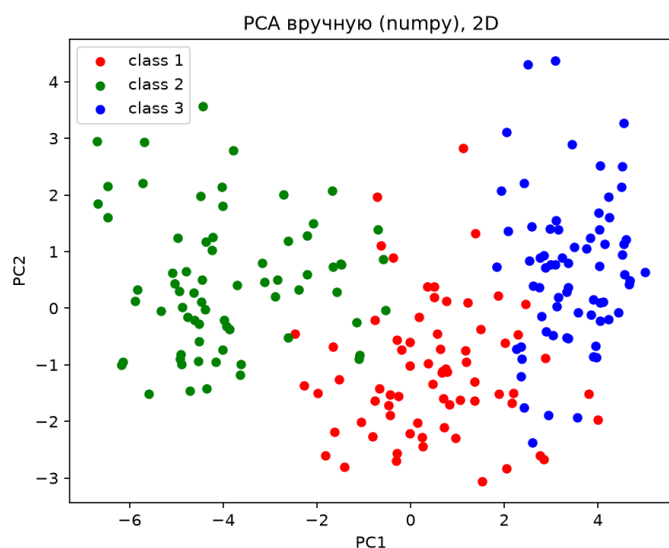
Результаты:

Потери (вручную, 2 компоненты): 0.70%

Потери (вручную, 3 компоненты): 0.13%

Потери (sklearn, 2 компоненты): 0.70%

Потери (sklearn, 3 компоненты): 0.13%



Вывод: Сравнительный анализ показал полную эквивалентность результатов ручной реализации PCA и встроенного модуля библиотеки sklearn. Сжатие исходного пространства признаков до 2 и 3 главных компонент сохранило ключевую структуру данных (потери информативности не превысили 0.13–0.70%), а на полученных графиках отчётливо прослеживается кластеризация различных типов зерна. Это доказывает высокую результативность метода PCA как для понижения размерности, так и для наглядной визуализации.